🎮 Extra — Score, Lives, and the Game Loop

Topic: The Frame Loop · Timers · HUD · Debugging · **Course:** Extra Worksheet · **Time:** about 60 minutes

> Keep these words handy: **frame**, **game loop**, **modulo**, **timer**, **blit**, **HUD**, **freeze**

Your shooter now has lives, invincibility, a game-over screen, and a score. Everything in a game happens **once per frame**: read input, move things, check hits, draw, flip. This worksheet is about what belongs inside that one pass — and what happens when something loops forever inside it.

> HOW TO USE THIS SHEET
>
> 1. Read on paper first. Do not open the IDE until section 5.
> 2. Every question has small steps under it. Do them in order.
> 3. If you are stuck, read the yellow box. It tells you where to look — not the answer.
> 4. Write something in every box. A wrong answer you can talk about beats an empty box.

1 · Predict 🔮

Read each block. Write what it does — just from reading.

> DO THIS FOR EVERY BLOCK
>
> 1. Say out loud what the first line sets up.
> 2. Count how many times the loop runs.
> 3. Track the variables that change. Write them down as they change.
> 4. Then write the output.

```
frames = 0
score = 0

for _ in range(31):
    if frames % 10 == 0:
        score += 10
    frames += 1

print(score)
```

Step 1 — Which values of `frames` make `frames % 10 == 0` true? List them.

Step 2 — How many are in your list?

Step 3 — So what number prints?

STUCK?

`%` gives the remainder after dividing. Try it with small numbers first: what is `0 % 10`, `5 % 10`, `10 % 10`?

```
enemies = [
    {"color": "RED", "speed": 10},
    {"color": "GREEN", "speed": 8},
]

for enemy in enemies:
    print(enemy["speed"])
```

Step 1 — On the first turn, what is the whole value of `enemy`?

Step 2 — What prints, line by line?

```
invincible_timer = 8

while invincible_timer > 0:
    print(invincible_timer % 6 < 3)
    invincible_timer -= 1
```

Step 1 — Write the timer values in order, from 8 down.

Step 2 — Under each one, write True or False.

Step 3 — In your game, that True/False decides whether the player is drawn. So what does the player look like?

```
enemy_h = 50

enemy_y = enemy_h
enemy_y2 = -enemy_h
```

Step 1 — In pygame, y = 0 is the top of the window. Where is y = 50? Where is y = −50?

Step 2 — Which one starts the enemy off-screen, ready to fall in?

2 · One Frame, In Order 🎞️

Your game loop does the same five jobs every frame.

> **HOW TO FILL THE TABLE**
>
> 1. Open your `main.py` and scroll to `while running:`.
> 2. Read down the loop. Every block belongs to one of the five jobs.
> 3. In each row, write what your game does there, and name one line from your file.

order	job	what happens in your game	one line from your file
1	read events		
2	move things		
3	check hits		
4	draw		
5	flip + tick		

`clock.tick(60)` runs once per frame. How many frames in 5 seconds?

`INVINCIBILITY_FRAMES = 60`. Using your answer above, how many seconds is the player safe after a hit?

> **WHAT FLIP() DOES**
>
> Pygame does not draw straight onto the window. Every `draw` and `blit` goes onto a **hidden page** that nobody can see yet. `pygame.display.flip()` swaps that hidden page onto the screen in one go — so the player never sees a half-drawn frame.

Why must `pygame.display.flip()` be the last job, not the first?

STUCK?

Think about what the screen would show if you flipped it before you drew this frame's enemies.

3 · Spot the Bug 🐛

These four come from your own game. Work through the steps for each one.

> **BUG METHOD — USE IT EVERY TIME**
>
> 1. Say what this code is **supposed** to do.
> 2. Read it one line at a time and say what it **actually** does.
> 3. Circle the exact line where those two stop matching.
> 4. Write the fixed code.
> 5. Say why your fix works.

Bug A — The window opens, then freezes. Nothing draws. Nothing responds.

```
## UPDATE SCORE ##
while True:
    if frames % 10 == 0:
        score += 10
    frames += 1
```

Step 1 — What is this block supposed to do, in one sentence?

Step 2 — What would end this `while` loop? Is there anything that can?

Step 3 — The lines after it never run. Name two things that stop happening.

Step 4 — Write the fixed code.

Step 5 — Why does your fix work?

STUCK?

The big `while running:` loop already repeats every frame. Ask yourself what this block still needs, once you are already inside something that repeats.

Bug B — Lives and score print on top of each other, so both are unreadable.

```
screen.blit(lives_surf, (10, 10))
screen.blit(scores_surf, (10, 10))
```

Step 1 — What do the two numbers in `(10, 10)` mean?

Step 2 — Write the fixed code. Put score on the right side of the window.

Step 3 — Why does your fix work?

STUCK?

You already have `WIDTH`. A position can be worked out from it instead of typed by hand.

Bug C — A bullet kills an enemy, but the enemy pops back near the top of the window instead of falling in from above.

```
if bullet.colliderect(enemy["rect"]):
    enemy["rect"].y = enemy['h']
    enemy["rect"].x = random.randint(0, WIDTH - enemy['w'])
```

Step 1 — Find the respawn lines in your ENEMY MOVEMENT section. Copy the `y` line here.

Step 2 — Put the two `y` lines side by side. What is different?

Step 3 — Write the fixed code, and say why your fix works.

Bug D — After GAME OVER, enemies keep jumping back to the top of the window.

```
if not game_over:
    ...
    player_rect = pygame.Rect(player_x, player_y, PLAYER_WIDTH, PLAYER_HEIGHT)
    ...

if invincible_timer > 0:
    invincible_timer -= 1
else:
    for enemy in enemies:
        if player_rect.colliderect(enemy["rect"]):
            lives -= 1
            enemy["rect"].y = -enemy["h"]
```

Step 1 — When `game_over` is `True`, which of these blocks still runs?

Step 2 — `player_rect` stops being updated. What value is it still holding?

Step 3 — Write the fixed code, and say why your fix works.

STUCK?

Ask which jobs should pause on the game-over screen, and which should keep going. Then look at where the `if not game_over:` block ends.

4 · Explain the Code 📖

Read your own lines. Answer from reading only.

```
if invincible_timer == 0 or invincible_timer % 6 < 3:
    pygame.draw.rect(screen, BLUE, (player_x, player_y, PLAYER_WIDTH,
PLAYER_HEIGHT))
```

Step 1 — When the timer is 0, is the player drawn? Say why.

Step 2 — For 6 frames in a row, how many frames is the player drawn, and how many not?

Step 3 — So what does the player look like right after being hit?

```
if player_x < 0:
    player_x = WIDTH - PLAYER_WIDTH

if player_y < 0:
    player_y = 0
```

The player walks off the left edge. Then off the top edge. Describe both, in your own words.

```
for bullet in bullets[:]:
```

Step 1 — What does `bullets[:]` give you that `bullets` does not?

Step 2 — Inside that loop you call `bullets.remove(bullet)`. What goes wrong without the colon?

```
enemies = [
    {"rect": ..., "color": RED, "speed": 10, "w": ENEMY_WIDTH, "h": ENEMY_HEIGHT},
]
```

Step 1 — You could have used six separate variables per enemy. What does the list of dictionaries give you instead?

Step 2 — Write the one line that adds a slow, wide, yellow enemy.

5 · Code It 💻

Now open `main.py` in the IDE at **app.english-coding.co.uk**. Do one task at a time. Run the game after every task.

Task 1 — Fix the freeze.

> **STEPS**
>
> 1. Find the `## UPDATE SCORE ##` block.
> 2. Decide which lines must run **once per frame**, and keep only those.
> 3. Score should stop climbing when `game_over` is `True` — put the block where that already happens.
> 4. Run it. Watch the score on screen.

> **YOU KNOW IT WORKS WHEN**
>
> the window responds to keys, the score climbs while you play, and it stops climbing on the GAME OVER screen.

Task 2 — Split the HUD.

> **STEPS**
>
> 1. Keep lives at the top left.
> 2. Move score to the top right. Work the x position out from `WIDTH`, do not type a number.
> 3. Change `WIDTH` to 700 and run. Then change it back.

> **YOU KNOW IT WORKS WHEN**
>
> both are readable, and the score is still on screen after you change `WIDTH`.

Task 3 — Points for kills.

> **STEPS**
>
> 1. Find the line where a bullet hits an enemy.
> 2. Add points there. Start with a flat number for every enemy.
> 3. Run it and shoot one enemy. Did the score jump by the right amount?
> 4. Now make a faster enemy worth more. Each enemy already carries its own `"speed"`.

> **YOU KNOW IT WORKS WHEN**
>
> shooting the green enemy and the magenta enemy give different points.

Write your plan here before you code.

STUCK ON ANY TASK?

Print the value you are unsure about — `print(score)`, `print(frames)` — run the game, and read the console. If a change breaks the game, undo it and make a smaller change.

6 · Explain Your Code 🎥

Record a short video on your phone explaining the code **you** wrote. Try to use these words: **frame**, **game loop**, **modulo**, **timer**, **blit**, **HUD**.

> 1. Run the game and lose one life. Point at the flicker and say what causes it.
> 2. Show the score line and explain when it goes up and when it stops.
> 3. Show one enemy dictionary and say what each key is for.
> 4. Show your fix for the freeze and explain what the old code did wrong.

Plan what you will say here before you record.

Submit ✅

Send this worksheet + your video or photo to teacher on KakaoTalk.